Санкт-Петербургский государственный университет
Кафедра системного программирования

Сравнение производительности LAGraph на CPU и spla на GPU

Вячеслав Кочергин, 23.Б11-мм

Санкт-Петербург
2026

Постановка задачи

Цель. Сравнить время работы штатных реализаций BFS, SSSP, подсчёта треугольников и PageRank в LAGraph на CPU и в spla на GPU на неориентированных графах различной топологии и объяснить наблюдаемые различия.

Исследовательский вопрос

При каких свойствах алгоритма и графа параллелизм GPU компенсирует запуски ядер OpenCL, синхронизацию с CPU и нерегулярный доступ к памяти?

Объект сравнения

LAGraph / CPU

- BFS: push/pull, эвристика Beamer, вектор родителей
- SSSP: delta-stepping, $\delta = 1$
- TC: SandiaDot, без пресортировки
- PageRank: GAP, останов по L_1

spla / GPU

- BFS: push/pull по плотности фронта, вектор уровней
- SSSP: релаксация изменившихся вершин
- TC: маскированное $LL^\top$; один work item на ребро маски
- PageRank: останов по L_2

Сравниваются именно готовые реализации по умолчанию, исходные коды примеров не были изменены.

Методика измерения

- Использована `spla-bench`: адаптирована под Python 3.10, унифицирован прогрев, подключены `perf` и flamegraph
- spla запускается только на GPU: `--run-cpu=false --run-ref=false --run-gpu=true`
- Первый запуск — прогрев; в статистику времени он не входит
- Основная оценка — медиана; отношение $R = T_{\mathrm{LAGraph}} / T_{\mathrm{spla}}$: $R > 1$ — быстрее spla, $R < 1$ — быстрее LAGraph
- На com-Orkut 21 запуск, на остальных графах — 40
- Источник BFS и SSSP: наибольшая компонента связности, степень ближайшая к медиане
- Время чтения Matrix Market не входит в время алгоритма

Экспериментальный стенд

Аппаратное обеспечение	
CPU	Intel Core i5-1135G7 2.40 ГГц, 4 ядра / 8 потоков
RAM	40 ГБ DDR4
L1d	192 КиБ, 4 экземпляра
L1i	128 КиБ, 4 экземпляра
L2	5 МиБ, 4 экземпляра
L3	8 МиБ, 1 экземпляр
GPU	Intel Iris Xe Graphics TigerLake-LP GT2 частота 400 МГц

Программное обеспечение	
OS	Ubuntu 22.04.5 LTS
Ядро	6.8.0-138-generic
`perf`	6.8.12
Python	3.10.12
CMake	4.4.2
GCC	11.4.0, `-O3`
GraphBLAS	6.1.4, C API 2.0
LAGraph	`aa150b1a`
spla	`b81d1e29`
OpenMP	4.5
OpenCL	3.0
intel-opencl-icd	22.14.22890-1

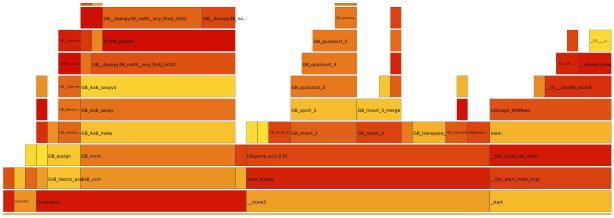
Перед измерениями скрипт подготовки стенда останавливает `irqbalance` и `thermald`, отключает подкачку, через `pyperf` фиксирует частоты CPU без Turbo Boost и ограничивает сэмплирование `perf` до 1 Гц, ослабляет ограничения на аппаратные счётчики и NMI watchdog, фиксирует частоту GPU и сбрасывает page cache. В выводе LAGraph зафиксировано 8 потоков.

Наборы данных

Граф	Вершины	Рёбра	Макс. степень	Класс
coAuthorsCiteseer	227 320	814 134	1 372	соавторство
coPapersDBLP	540 486	15 245 729	3 299	соавторство
belgium_osm	1 441 295	1 549 970	10	дорожный
roadNet-CA	1 971 281	2 766 607	12	дорожный
road_central	14 081 816	16 933 413	8	дорожный
rgg_n_2_22_s0	4 194 304	30 359 198	36	случайный геометрический
rgg_n_2_23_s0	8 388 608	63 501 393	40	случайный геометрический
com-Orkut	3 072 441	117 185 083	33 313	социальный

- Все восемь графов — неориентированные
- Matrix Market хранит одну треугольную часть; обе библиотеки восстанавливают полную симметричную матрицу
- Набор покрывает узкие длинные обходы, широкие социальные фронты, однородные геометрические графы и экстремальную неоднородность степеней

BFS: LAGraph быстрее на пяти графах из восьми



Flamegraph LAGraph, `road_central`.

Граф	CPU мс	GPU мс	R	IPC	LLC %	ин/р
coAuthors	37.3	38.8	**0.96**	0.46	25.3	150
coPapers	116.5	104.6	**1.11**	0.99	48.7	98
belgium	377.5	1362.6	**0.28**	0.68	29.7	360
roadNet	595.3	780.5	**0.76**	0.82	19.2	328
road-central	5500.7	5772.1	**0.95**	0.87	26.9	341
RGG22	3930.9	3427.2	**1.15**	0.80	40.5	241
RGG23	8396.5	6144.9	**1.37**	0.64	40.8	230
Orkut	188.9	274.5	**0.69**	1.27	34.8	99
медиана	486.4	1071.6	0.96	0.81	32.3	235

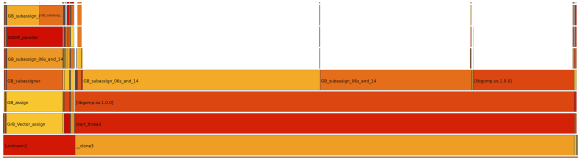
IPC — инструкции на циклы. LLC — промахи последнего уровня кэша. ин/р — инструкции на ребро.

Обоснование

- Главная операция LAGraph BFS — `GB_AxB_saxpy3`: умножение вектора фронта на матрицу смежности
- spla проигрывает на дорожных графах из-за узкого фронта. `coAuthorsCiteseer` мал, поэтому накладные расходы spla не успевают окупиться. На com-Orkut spla проигрывает из-за высокой неоднородности степеней и слабого GPU
- LAGraph проигрывает там, где высок LLC: `coPapersDBLP` $48{,}7\,\%$, RGG22 $40{,}5\,\%$, RGG23 $40{,}8\,\%$.

Вывод. LAGraph быстрее на 5 графах из 8. spla быстрее на `coPapersDBLP` и обоих RGG, максимум $R = 1{,}37$. На `belgium_osm` разрыв наибольший: GPU медленнее в $3{,}6$ раза.

SSSP: стоимость создаёт обновление глобальных структур



Flamegraph LAGraph, `road_central`.

Граф	CPU мс	GPU мс	R	IPC	LLC %	ср. ст.	ин/р
coAuthors	130.9	45.8	**2.86**	0.80	19.5	7.16	578
coPapers	354.1	288.0	**1.23**	0.96	36.7	56.41	173
belgium	6431.6	1368.2	**4.70**	1.38	11.0	2.15	41 600
roadNet	4486.6	963.7	**4.66**	0.53	25.3	2.81	10 757
road-central	176 795	6769.9	**26.11**	1.48	49.9	2.41	99 893
RGG22	35 157	7525.6	**4.67**	0.76	49.8	14.48	5 624
RGG23	76 169	14 313	**5.32**	1.18	54.5	15.14	7 132
Orkut	1675.6	1482.1	**1.13**	1.08	28.3	76.28	245
медиана	5459.1	1425.1	4.66	1.02	32.5	10.82	6 378

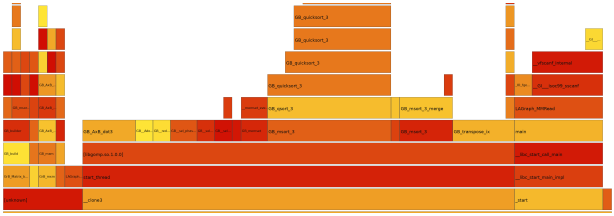
ср. ст. — средняя степень.

Обоснование

- Главная операция LAGraph SSSP — `GB_subassign_06s_and_14`: обновление глобальных структур
- На `road_central` в среднем на запуск $1{,}7 \cdot 10^{12}$ инструкций, IPC $1{,}48$, промахи ветвлений $0{,}056\,\%$, LLC $49{,}9\,\%$: разрыв задаёт объём обновлений глобальных структур при $\delta = 1$
- При низкой средней степени кратчайшие пути длинные, шагов обновления глобальных структур больше, поэтому растут ин/р и выигрыш GPU

Вывод. spla быстрее на всех 8 графах: от $R = 1{,}13$ на com-Orkut до $26{,}11$ на `road_central`.

Подсчёт треугольников: независимые пересечения выгодны GPU



Flamegraph LAGraph, `road_central`.

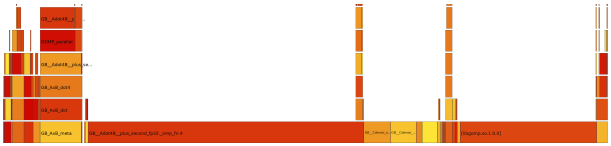
Граф	CPU мс	GPU мс	R	IPC	LLC %	ин/р
coAuthors	88.7	14.9	**5.96**	0.81	29.3	114
coPapers	1362.6	949.2	**1.44**	1.61	54.5	191
belgium	48.8	29.3	**1.67**	1.61	49.5	122
roadNet	94.5	40.6	**2.32**	1.46	53.8	115
road-central	717.5	250.2	**2.87**	1.32	58.1	123
RGG22	1003.0	407.9	**2.46**	1.39	60.2	107
RGG23	2176.1	871.5	**2.50**	1.38	63.2	108
Orkut	56 608	501.3	**112.92**	0.84	59.1	603
медиана	860.3	329.1	2.48	1.38	56.3	118

Обоснование

- Главная операция LAGraph TC — `GB_AxB_dot3` / `GB__Adot3B__plus_pair_int64`: маскированное пересечение строк
- У spla один work item — одно ребро маски. Независимые пересечения хорошо загружают GPU
- Больше всего выигрыш spla там, где много соседей у малого количества вершин, а на `coPapers` выигрыш мал, потому что степени высокие, но размазаны по большому количеству вершин

Вывод. spla быстрее на всех 8 графах. На семи графах разрыв $1{,}44$–$5{,}96$ раза, на com-Orkut — в $112{,}92$ раза.

PageRank: регулярный SpMV выгоден GPU



Flamegraph LAGraph, `road_central`.

Граф	CPU мс	GPU мс	R	IPC	LLC %	ин/р
coAuthors	232.0	8.5	**27.42**	0.35	5.2	284
coPapers	751.1	29.6	**25.36**	1.10	5.2	207
belgium	467.1	37.9	**12.34**	0.71	33.9	572
roadNet	664.5	79.3	**8.37**	0.66	39.1	457
road-central	3805.8	519.4	**7.33**	0.78	35.7	542
RGG22	1314.7	162.0	**8.12**	1.14	57.4	189
RGG23	2459.8	407.0	**6.04**	1.31	58.0	188
Orkut	8533.3	502.1	**17.00**	0.73	22.9	353
медиана	1032.9	120.7	10.35	0.76	34.8	318

Обоснование

- Главная операция LAGraph PageRank — `GB_AxB_dot4`: умножение матрицы смежности на плотный вектор рангов
- `perf`: медианный IPC $0{,}76$; на `coAuthorsCiteseer` IPC $0{,}35$
- При маленьком векторе рангов и неоднородных степенях выигрыш GPU больше

Вывод. spla быстрее на всех 8 графах в $6{,}04$–$27{,}42$ раза. Часть разрыва методическая: разные нормы, разное число итераций и разное положение нормализации относительно таймера.

Итоги

1. **BFS.** LAGraph быстрее на 5 графах из 8, spla на 3. R от $0{,}28$ до $1{,}37$
2. **SSSP.** spla быстрее на всех 8 графах. R от $1{,}13$ до $26{,}11$
3. **TC.** spla быстрее на всех 8 графах. R от $1{,}44$ до $112{,}92$
4. **PageRank.** spla быстрее на всех 8 графах. R от $6{,}04$ до $27{,}42$

Главный вывод

Из 32 сравнений spla/GPU быстрее в 27 случаях. Все пять побед LAGraph — BFS. В SSSP, подсчёте треугольников и PageRank spla быстрее на каждом графе.